

Module 4 Assignment III

Student A: Diya Jain U81673988

Student B: Suraj Kumar U95327637

Section I:

1. **Draft Answer: Diya Jain.**

Suraj Kumar

RISC-V Code	Comments
binarySearch: addi sp, sp, -20 sw a3, 16 (sp) sw a2, 12 (sp) sw a1, 8 (sp) sw a0, 4 (sp) sw ra, 0 (sp) blt a2, a1, L1 add t0, a1, a2 srai t0, t0, 1 slli t1, t0, 2 add t2, t1, a0 lw t3, 0(t2) blt a3, t3, Rt1 blt t3, a3, Rt2 Base: add a0, x0, t0 addi sp, sp, 20 jr ra Rt1: addi a2, t0, -1 jal binarySearch Rt2: addi a1, t0, 1 jal binarySearch L1: addi a0, x0, -1 jr ra	 #allocating space in memory #a3 = target variable defined #a2 = high #a1 = low #a0 = nums[] # a2 > a1 and if not jump to L1 #t0 = high + low # main function formula # t1 = t0 *4 # t2 = t1 + nums[] # nums[mid] # target < Nums[mid]. If false, jump to Rt1 # Nums[mid] < target. If false, jump to Rt2 # a0 = t0 # Deallocating space in memory # return address # a2 = t0 -1 # jump and link # a2 = t0 +1 # jump and link # return address Judgement: The code is clean and working.

Discussion: This question is related to recursion. We will be using stack to store the pointers related to the parameters. In the binarySearch we will store and use the function parameters. The base function will release the stack. During discussion, I did not use stack pointers, but my partner helped me in using the stack pointers.

Final Answer:

binarySearch:
 addi sp, sp, -20
 sw a3, 16 (sp)
 sw a2, 12 (sp)
 sw a1, 8 (sp)
 sw a0, 4 (sp)

```

sw ra, 0(sp)
blt a2, a1, L1
add t0, a1, a2
srai t0, t0, 1
slli t1, t0, 2
add t2, t1, a0
lw t3, 0(t2)
blt a3, t3, Rec1
blt t3, a3, Rec2

```

Base:

```

add a0, x0, t0
addi sp, sp, 20
jr ra

```

Rt1:

```

addi a2, t0, -1
jal binarySearch

```

Rt2:

```

addi a1, t0, 1
jal binarySearch

```

L1:

```

addi a0, x0, -1
jr ra

```

2. Draft Answer: Diya Jain

Suraj Kumar

RISC-V	Comments
<pre> swap: addi t2, a0, 0 addi a0, a1, 0 addi a1, t2, 0 jr ra reverse: addi sp, sp, -16 sw a2, 12(sp) sw a1, 8(sp) sw a0, 4(sp) sw ra, 0(sp) blt a1, a2, Rt base: addi sp, sp, +16 j Exit rec: addi a0, a1, a0 addi a1, a2, a0 jal swap lw a0, 4(sp) lw a1, 8(sp) lw a2, 12(sp) addi a1, a1, 1 addi a2, a2, -1 jal reverse Exit: </pre>	<pre> #t2 = *x #*x = *y #*y = t2 #Jump statement # allocating space in memory # h = a2 # c = a1 # string[] = a0 # l < h. If false, jump to Rt # deallocating space # jump to Exit # a0 = str[l] # a1 = str{h} #jump # Allocating space between a0, a1 and a2 # a2 = h-1 # a1 = a2 -1 # jump to reverse </pre> Judgement: The code is clean and correct.

Discussion: This a question which contains the concept of recursion. We have first written the code for the function swap. In the swap function to store register values in a stack. The base condition is used to deallocate the stack. The function rec will do the rest of the work in the code.

Final Answer:

swap:

```
addi t2, a0, 0
addi a0, a1, 0
addi a1, t2, 0
jr ra
```

reverse:

```
addi sp, sp, -16
sw a2, 12(sp)
sw a1, 8(sp)
sw a0, 4(sp)
sw ra, 0(sp)
blt a1, a2, Rec
```

base:

```
addi sp, sp, +16
j exit
```

rec:

```
addi a0, a1, a0
addi a1, a2, a0
jal swap
lw a0, 4(sp)
lw a1, 8(sp)
lw a2, 12(sp)
addi a1, a1, 1
addi a2, a2, -1
jal reverse
```

Exit:

Section II

1. **Draft Answer:** Suraj Kumar

Diya Jain

RISC-V Code	Comments
getNextValue:	
addi sp, sp, -4	# allocating space in memory
sw ra, 0. (sp)	# distributing space to ra
addi t0, a0, 0	# t0 = a0
andi t1, t0, 1	#t1 = t0 +1
bne t1, zero, else	# t1 != 0 else jump to else
srai t2, t1, 1	# t2 = t1 *2
j exit	# jump to exit
else:	
slli t2, t1, 1	# t2 = t1 /2
add t2, t2, t0	# t2 = t2 +t0
addi t2, t2, 1	# t2 = t2 +1
exit:	
addi a0, t0, 0	# a0 = t0 +0
addi sp, sp, 4	# dellocating memory
jr ra	

<pre> HaistoneSeq: addi t4, zero, 1 bne t3, t4, else li a7, 1 add a0, t0, zero ecall else: li a7, 1 add a0, t0, zero ecall jal getNextValue lw ra, 0 (sp) addi a0, t3, 0 addi sp, sp, 4 jr ra exit: </pre>	<pre> #t4 = 1 # t3 != t4 else, jump to else #load integer # a0 = t0 #ecall func # load integer # a0 = t0 # jump to getNextValue # load word # a0 = t3 # deallocating # jump </pre> Judgement: The code is correct and cleanly written.
--	--

2. Draft Answer: Suraj Kumar

Diya Jain

RISC-V Code	Comments
<pre> findGCD: addi sp, sp, -12 sw a1, 8 (sp) sw a0, 4 (sp) sw ra, 0 (sp) while: beq a0, a1, Exit blte a0, a1, else sub a0, a0, a1 addi sp, sp, 12 jal findGCD jr ra j Jump else: sub a1, a1, a0 jal findGCD jr ra j Jump Exit: lw ra, 0 (sp) lw a0, 4 (sp) lw a1, 8 (sp) addi sp, sp, 12 addi a0, a0, 0 jr ra Jump: j while </pre>	<pre> #Allocating space # store word function to distribute space in all 3 # a0 = a1 else, jump to Exit #a0 <= a1 else, jump to else # a0 = a0- a1 # dellocating space #Jump and link #jump to Jump #a1 = a1- a0 #jump and link # jump to Jump # load word function to get spaces from all 3 # dellocating space # a0 = a0 # jump #jump to while </pre> Judgement: The code is correct and cleanly written.